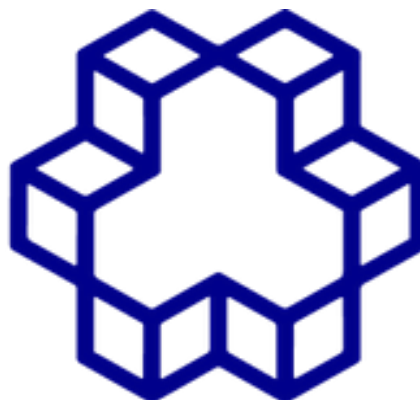




گزارش پروژه نهایی یادگیری تقویتی

حل مسئله‌ی ماز پویا با استفاده از Value Iteration، Q-Learning و $SARSA(\lambda)$

استاد درس: دکتر خواسته

دانشجو: ریحانه مرادی 40413824

موضوع پروژه: پیاده‌سازی و مقایسه‌ی الگوریتم‌های یادگیری تقویتی در محیط ماز پویا

تابستان 1405

1. مقدمه

در این پروژه، هدف اصلی طراحی و پیاده‌سازی یک محیط ماز پویا و سپس استفاده از چند الگوریتم مهم یادگیری تقویتی برای حل مسئله‌ی مسیریابی در این محیط بوده است. عامل (Agent) باید از نقطه‌ی شروع حرکت کرده، در صورت نیاز کلید را پیدا کند و در نهایت به هدف برسد. در این مسیر، عامل با موانع، حرکت‌های احتمالی، جریمه‌ها و تغییرات محیطی مواجه می‌شود.

برای حل این مسئله، سه الگوریتم اصلی پیاده‌سازی و بررسی شده‌اند:

1. Value Iteration

2. Q-Learning

3. SARSA(λ)

همچنین در این پروژه بخش‌های تکمیلی مانند تولید خودکار ماز، رابط گرافیکی (GUI)، آزمایش‌های تحلیلی، یادگیری انتقالی (Transfer Learning) و تست خودکار نیز در نظر گرفته شده‌اند تا پروژه از نظر ساختار، عملکرد و ارزیابی کامل باشد.

2. هدف پروژه

هدف این پروژه بررسی توانایی الگوریتم‌های مختلف یادگیری تقویتی در حل یک مسئله‌ی تصمیم‌گیری ترتیبی در محیطی غیرقطعی و نسبتاً پیچیده است. به طور مشخص، اهداف پروژه شامل موارد زیر بوده‌اند:

- طراحی یک محیط ماز با قابلیت تولید تصادفی و بازتولیدپذیر
- تعریف حالت‌ها، اعمال، پاداش‌ها و مدل انتقال در قالب یک مسئله‌ی MDP
- پیاده‌سازی الگوریتم‌های مدل‌مبنا و مدل‌آزاد
- مقایسه‌ی رفتار الگوریتم‌ها در شرایط مختلف
- بررسی اثر نوع تابع پاداش بر روند یادگیری
- افزودن قابلیت یادگیری انتقالی بین دو محیط مشابه
- فراهم کردن امکان مشاهده و تحلیل عملکرد عامل از طریق رابط گرافیکی و آزمایش‌های دسته‌ای

3. ساختار کلی پروژه

پروژه به چند بخش اصلی تقسیم شده است که هر کدام وظیفه‌ی مشخصی را بر عهده دارند:

3.1 پوشه agents

در این بخش، الگوریتم‌های یادگیری تقویتی پیاده‌سازی شده‌اند. هر فایل مربوط به یک عامل یادگیرنده است:

- value_iteration.py
- q_learning.py
- sarsa_lambda.py

3.2 پوشه environments

این بخش مربوط به محیط پروژه است و شامل تولید ماز و منطق حرکت عامل در محیط می‌شود:

- generator.py برای تولید ماز
- maze.py برای تعریف محیط، حالت‌ها، انتقال‌ها و پاداش‌ها

3.3 پوشه gui

این بخش رابط گرافیکی پروژه را پیاده‌سازی می‌کند و به کاربر اجازه می‌دهد محیط را مشاهده کرده، الگوریتم‌ها را اجرا کند و رفتار عامل را ببیند.

3.4 پوشه experiments

در این قسمت اجرای آزمایش‌ها، بارگذاری تنظیمات و ذخیره‌ی نتایج انجام می‌شود.

3.5 پوشه transfer

این بخش مربوط به یادگیری انتقالی است؛ یعنی انتقال دانش یادگرفته‌شده از یک ماز به ماز دیگر.

3.6 پوشه tests

برای اطمینان از صحت عملکرد بخش‌های مختلف، تعدادی تست خودکار برای محیط، عامل‌ها، تولیدکننده‌ی ماز و یادگیری انتقالی نوشته شده است.

4. تعریف محیط ماز

محیط این پروژه یک ماز دوبعدی است که عامل در آن حرکت می‌کند. وضعیت محیط به گونه‌ای تعریف شده که تنها موقعیت مکانی عامل کافی نیست، بلکه برخی ویژگی‌های دیگر نیز در تعریف حالت نقش دارند. برای مثال:

- موقعیت عامل در ماز
- داشتن یا نداشتن کلید
- موقعیت هدف
- موانع و دیوارها
- خانه‌های دارای جریمه

در این محیط، اعمال ممکن عامل معمولاً شامل چهار حرکت اصلی است:

- بالا
- پایین
- چپ
- راست

اما حرکت‌ها کاملاً قطعی نیستند؛ به این معنا که ممکن است عامل عملی را انتخاب کند اما به دلیل **احتمالی بودن انتقال**، حرکت واقعی با احتمال مشخصی متفاوت باشد. این ویژگی باعث می‌شود مسئله به شرایط واقعی‌تر نزدیک شود و برای الگوریتم‌ها چالش بیشتری ایجاد کند.



5. تولید ماز و وابستگی به شماره دانشجویی

یکی از بخش‌های مهم پروژه، تولید محیط به‌صورت بازتولیدپذیر و وابسته به شماره دانشجویی بوده است. بر اساس صورت پروژه، اندازه‌ی ماز و بذر تصادفی از روی شماره دانشجویی استخراج می‌شود. برای شماره دانشجویی:

40413824

رقم یکی مانده به آخر برابر 2 است. بنابراین:

• $\text{base_seed} = 2$

• $\text{maze_size} = 15 + (2 \% 4) = 17$

در نتیجه، اندازه‌ی پیش‌فرض ماز در این پروژه 17×17 در نظر گرفته شده است. این کار باعث می‌شود تولید محیط برای هر دانشجو منحصر به فرد و در عین حال تکرارپذیر باشد.

6. تابع پاداش

در این پروژه، دو نوع تابع پاداش در نظر گرفته شده است:

6.1 پاداش شکل‌دهی‌شده (Shaped Reward)

در این حالت علاوه بر پاداش نهایی رسیدن به هدف، برای برخی وضعیت‌ها یا حرکت‌ها نیز پاداش یا جریمه‌ی کمکی داده می‌شود. این کار باعث می‌شود عامل در طول مسیر بازخورد بیشتری دریافت کند و معمولاً سریع‌تر یاد بگیرد.

6.2 پاداش تنک (Sparse Reward)

در این حالت عامل فقط در شرایط خاص، مثل رسیدن به هدف، پاداش معنی‌دار دریافت می‌کند. این مدل سخت‌تر است و معمولاً یادگیری در آن زمان بیشتری می‌برد، اما برای بررسی توان واقعی الگوریتم‌ها مفید است.

7. الگوریتم Value Iteration

الگوریتم Value Iteration یک روش مدل‌مبنا است. در این روش فرض می‌کنیم مدل محیط، یعنی احتمال انتقال بین حالت‌ها و مقدار پاداش‌ها، در دسترس است. سپس با استفاده از معادله‌ی بلمن، مقدار هر حالت به صورت تکراری به‌روزرسانی می‌شود تا به مقدار بهینه نزدیک شود.

پس از همگرایی مقدارها، سیاست بهینه از روی آن‌ها استخراج می‌شود؛ یعنی در هر حالت، عملی انتخاب می‌شود که بیشترین ارزش مورد انتظار را ایجاد کند.

مزایای Value Iteration

- محاسبه‌ی مستقیم سیاست بهینه
- مناسب برای محیط‌های کوچک یا با مدل معلوم
- امکان تحلیل دقیق مقدار حالت‌ها

محدودیت‌ها

- نیاز به دانستن مدل کامل محیط
- افزایش شدید هزینه‌ی محاسباتی در فضاهاى حالت بزرگ

8. الگوریتم Q-Learning

الگوریتم **Q-Learning** یک روش مدل‌آزاد و **off-policy** است. این الگوریتم بدون نیاز به دانستن مدل محیط، تنها از طریق تجربه و تعامل با محیط یاد می‌گیرد.

در این روش، یک جدول Q نگهداری می‌شود که برای هر زوج حالت-عمل، ارزش تخمینی آن را نشان می‌دهد. عامل با تکرار تجربه‌ها و استفاده از قاعده‌ی بهروزرسانی Q-Learning، به تدریج این مقادیر را اصلاح می‌کند.

ویژگی‌های Q-Learning

- عدم نیاز به مدل انتقال
- مناسب برای یادگیری از تجربه
- سادگی پیاده‌سازی
- توانایی یادگیری سیاست خوب در بسیاری از مسائل

نکته مهم

Q-Learning برای اکتشاف و بهره‌برداری معمولاً از روش‌هایی مانند **epsilon-greedy** استفاده می‌کند تا بین امتحان کردن عمل‌های جدید و انتخاب عمل‌های خوب قبلی تعادل برقرار شود.

9. الگوریتم $SARSA(\lambda)$

الگوریتم $SARSA(\lambda)$ نیز یک روش مدل آزاد است، اما برخلاف Q-Learning، از نوع **on-policy** محسوب می‌شود. در این روش، بهروزرسانی بر اساس همان سیاستی انجام می‌شود که عامل در حال استفاده از آن است.

وجود پارامتر λ در $SARSA(\lambda)$ باعث استفاده از **eligibility traces** می‌شود. این مکانیزم کمک می‌کند اثر پاداش‌ها و خطاهای یادگیری به چندین حالت و عمل قبلی منتقل شود، نه فقط آخرین گام. در نتیجه، در برخی مسائل یادگیری سریع‌تر و روان‌تر انجام می‌شود.

مزایای $SARSA(\lambda)$

- انتشار سریع‌تر اطلاعات پاداش
- مناسب برای مسائل ترتیبی
- رفتاری نرم‌تر در برخی محیط‌های تصادفی

تفاوت با Q-Learning

- Q-Learning بیشتر به سمت سیاست بهینه‌ی حریصانه حرکت می‌کند.
- $SARSA$ رفتار واقعی عامل تحت سیاست جاری را یاد می‌گیرد.

10. رابط گرافیکی (GUI)

برای ساده‌تر شدن بررسی و نمایش عملکرد پروژه، یک رابط گرافیکی طراحی شده است. این رابط به کاربر اجازه می‌دهد:

- الگوریتم مورد نظر را انتخاب کند
- آموزش را شروع یا متوقف کند
- حرکت عامل را در ماز مشاهده کند
- سیاست یادگرفته‌شده را به‌صورت دیداری بررسی کند
- در صورت نیاز عامل را به‌صورت دستی کنترل کند
- اجرای خودکار بر اساس سیاست حریصانه را فعال کند

رابط گرافیکی باعث شده پروژه از حالت صرفاً کدنویسی خارج شود و به‌صورت تعاملی قابل مشاهده و تحلیل باشد.

11. اجرای خودکار سیاست (Autoplay)

یکی از قابلیت‌های مهم اضافه‌شده به پروژه، اجرای خودکار عامل بر اساس سیاست یادگرفته‌شده است. در این بخش، عامل پس از یادگیری می‌تواند به‌صورت خودکار و بدون دخالت کاربر در محیط حرکت کند. این ویژگی به‌ویژه برای الگوریتم‌های مدل‌آزاد مفید است، زیرا نشان می‌دهد جدول Q یا سیاست نهایی تا چه اندازه توانسته مسیر مناسب را یاد بگیرد. همچنین برای دیباگ و تحلیل رفتار عامل در محیط نقش مهمی دارد.

12. آزمایش‌ها و تحلیل نتایج

در بخش آزمایش‌ها، امکان اجرای مجموعه‌ای از سناریوهای از پیش تعریف‌شده فراهم شده است. این آزمایش‌ها برای مقایسه‌ی الگوریتم‌ها از نظر معیارهایی مانند موارد زیر استفاده می‌شوند:

- سرعت همگرایی
- مجموع پاداش دریافتی
- تعداد قدم‌های لازم تا رسیدن به هدف
- پایداری عملکرد در محیط‌های مختلف
- تأثیر نوع تابع پاداش
- عملکرد در شرایط انتقال دانش

همچنین نتایج به‌صورت فایل‌های خروجی ذخیره می‌شوند تا بعداً قابل تحلیل باشند. این فایل‌ها می‌توانند شامل مواردی مانند تنظیمات آزمایش، مقادیر عملکرد، آمار آموزشی، جدول‌های Q و تصاویر محیط باشند.

13. یادگیری انتقالی (Transfer Learning)

در این پروژه، فقط حل یک محیط ثابت مد نظر نبوده است؛ بلکه قابلیت **یادگیری انتقالی** نیز پیاده‌سازی شده است. ایده‌ی اصلی این بخش آن است که عامل ابتدا در یک محیط مبدأ آموزش ببیند و سپس دانش به‌دست‌آمده را در یک محیط مقصد که کمی تغییر کرده است به کار گیرد.

این تغییرات ممکن است شامل موارد زیر باشند:

- تغییر بخشی از ساختار ماز
- جابه‌جایی کلید

- جابه‌جایی هدف
 - تغییر بعضی مسیرها یا موانع
- هدف از این بخش بررسی این است که آیا استفاده از دانش قبلی باعث سریع‌تر شدن یادگیری در محیط جدید می‌شود یا خیر.

14. تست و ارزیابی صحت کد

برای اطمینان از درست بودن عملکرد بخش‌های مختلف پروژه، مجموعه‌ای از تست‌ها نوشته شده است. این تست‌ها بخش‌های مختلف زیر را پوشش می‌دهند:

- درستی تولید ماز
 - عملکرد صحیح محیط و انتقال‌ها
 - صحت اجرای الگوریتم‌ها
 - درستی اجزای مربوط به یادگیری انتقالی
- استفاده از تست باعث می‌شود خطاهای منطقی در مراحل اولیه شناسایی شوند و پایداری پروژه افزایش یابد.

15. جمع‌بندی عملکرد پروژه

در این پروژه یک سامانه‌ی نسبتاً کامل برای بررسی الگوریتم‌های یادگیری تقویتی طراحی و پیاده‌سازی شد. این سامانه شامل محیط ماز، چند الگوریتم استاندارد، رابط گرافیکی، آزمایش‌های تحلیلی، یادگیری انتقالی و تست خودکار است. به طور خلاصه، کارهای انجام‌شده در پروژه شامل موارد زیر بوده است:

- طراحی محیط ماز پویا
- تعریف حالات، اعمال و پاداش‌ها
- پیاده‌سازی Value Iteration
- پیاده‌سازی Q-Learning
- پیاده‌سازی SARSA(λ)
- ایجاد تولیدکننده‌ی ماز وابسته به seed
- اعمال وابستگی به شماره دانشجویی برای تعیین اندازه و seed

- افزودن رابط گرافیکی برای مشاهده و کنترل اجرای عامل
- پیاده‌سازی اجرای خودکار بر اساس سیاست یادگرفته‌شده
- ساخت بخش آزمایش‌ها و تنظیمات JSON
- پیاده‌سازی یادگیری انتقالی بین دو محیط
- نوشتن تست برای ارزیابی صحت عملکرد بخش‌های مختلف

16. نتیجه‌گیری

این پروژه نشان داد که الگوریتم‌های مختلف یادگیری تقویتی، رفتارها و ویژگی‌های متفاوتی در حل یک مسئله‌ی مشابه دارند.

- **Value Iteration** در صورت داشتن مدل محیط، رامحلی دقیق و تحلیلی ارائه می‌دهد.
 - **Q-Learning** بدون نیاز به مدل، از طریق تجربه یاد می‌گیرد و برای مسائل عملی بسیار مهم است.
 - **SARSA(λ)** نیز با استفاده از eligibility traces می‌تواند یادگیری را در برخی شرایط بهبود دهد.
- همچنین وجود ماژول‌های کمکی مانند رابط گرافیکی، آزمایش‌های دسته‌ای و یادگیری انتقالی، پروژه را از یک پیاده‌سازی ساده فراتر برده و آن را به یک چارچوب کامل برای مطالعه و مقایسه‌ی الگوریتم‌های یادگیری تقویتی تبدیل کرده است.